Model-Based Reinforcement Learning: From Learned Dynamics to Differentiable Policy Optimization

Dyna, PETS, Dreamer, SHAC, and SuperMimic | Foundation of Embodied AI

2026-10-28

Contents

1. Why model-based reinforcement learning? . . . 2
2. Learning a world model . . . 2
3. Three ways to use the world model . . . 3
4. Deriving pathwise gradients and BPTT . . . 4
5. Short-horizon actor–critic and model bias . . . 5
6. SuperMimic: a shared dynamics model for whole-body tracking . . . 5
7. Contact gradients in manipulation . . . 6
8. Reproducible training and evaluation . . . 6
9. Study route and references . . . 7

Course home ·简体中文 ·SuperMimic ·SHAC

Item	Details
Topic	Model-Based RL
Revision	2026-10-28
License	CC BY-NC-SA 4.0 ·Xiong Zeng and contributors

1. Why model-based reinforcement learning?

> **Scope:** A missing link between the course's PPO, SAC, robot dynamics, MuJoCo and motion tracking tutorials. This is a conceptual and mathematical tutorial; it does not present unverified code or invented experimental results.

In a Markov decision process, the environment evolves according to $P(s_{t+1} \mid s_t, a_t)$. The objective is

$$J(\pi) = \mathbb{E}_{P,\pi}\left[\sum_{t=0}^{T-1} \gamma^t r(s_t, a_t)\right].$$

Model-free RL (such as PPO and SAC) updates a policy or value function from observed transitions and rewards without explicitly predicting the next state. **Model-based RL (MBRL)** additionally uses a known or learned transition model $\hat{P}$ (or deterministic f_ω) to forecast, plan, or optimize.

Three decisions are independent: whether a dynamics model exists, whether it is differentiable, and whether the actor is optimized by backpropagating through that model. MPC can optimize actions through a nondifferentiable model by sampling. Dyna can generate synthetic data for SAC. SuperMimic differentiates through a learned surrogate to obtain a pathwise policy gradient. **The high-fidelity physics engine itself need not be differentiable**.

1.1 Data efficiency in robotic control

Real robot interactions require wall-clock time, resets and safe handling of contact. Massively parallel simulation reduces sample cost, but large and diverse tracking tasks still require substantial compute. A shared learned model can reuse invariant robot and contact dynamics across many task commands rather than relying on new expensive physics rollouts for every local policy improvement.

This does **not** imply that the model is more accurate than the simulator. Approximation error, distribution shift and model exploitation are first-class risks.

2. Learning a world model

2.1 State, action, observation, context

A useful robot-learning contract is

$$s_t = (q_t, \dot{q}_t, x_t^{obj}, v_t^{obj}), \quad a_t = \text{applied joint command}, \quad o_t = h(s_t) + \eta_t.$$

The optional object pose/twist and domain context ξ (inertia, friction, actuator gains) determine what can be learned. **Privileged simulator state s_t and deployable observation o_t are different.** A privileged teacher can train the model, but the deployed actor must respect its actual sensors.

The replay buffer contains

$$\mathcal{B} = \{(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1}, g_t, \xi)\}.$$

Store actions **actually applied** after noise and saturation. Distinguish failure termination, successful completion and time-limit truncation. Never form autoregressive training sequences across an episode reset.

2.2 One-step and autoregressive multi-step fitting

A state model predicts

$$\hat{s}_{t+1} = f_\omega(\hat{s}_t, a_t, \xi), \qquad \hat{s}_t = s_t.$$

For multi-step fitting, recursively feed predictions into the next input, using actions recorded in the replay buffer:

$$\hat{s}_{t+j+1} = f_\omega(\hat{s}_{t+j}, a_{t+j}, \xi), \qquad \mathcal{L}_{dyn} = \mathbb{E}_{\mathcal{B}} \sum_{j=1}^{K} \|\hat{s}_{t+j} - s_{t+j}\|^2_{W_s}.$$

Autoregressive fitting exposes error accumulation hidden by one-step teacher forcing. Rotation losses must account for unit-quaternion normalization and the equivalence $q \sim -q$ or use a suitable rotation-geodesic metric.

If reward, noise-free observation and termination are also modeled, one may use

$$\mathcal{L}_{WM} = \mathcal{L}_{dyn} + \lambda_r \mathcal{L}_{reward} + \lambda_o \mathcal{L}_{observation} + \lambda_d \mathcal{L}_{termination}.$$

Binary cross entropy suits termination probabilities. When a reward or observation is exactly computable from state, an analytic head can replace its learned counterpart.

2.3 Uncertainty and out-of-distribution control

A world model can be precise on replay-buffer states but wrong where an optimized actor subsequently visits. Report held-out multi-step accuracy, state coverage, ensemble disagreement and realized simulator performance. PETS uses probabilistic ensembles and trajectory sampling to estimate uncertainty; a single deterministic surrogate is cheaper but especially needs frequent refresh and conservative optimization.

3. Three ways to use the world model

Family	Examples	Role of model	How actions improve
Synthetic experience / Dyna	Dyna, MBPO-style learning	Imagined transitions for value/policy updates	SAC, temporal difference or similar updates
Online planning	PETS, CEM, MPPI, MPC	Forecast candidate action sequences	Search / sampling at each control step
Differentiable policy learning	SHAC, SuperMimic	Differentiable transition graph	BPTT on policy parameters
Latent imagination	Dreamer, some TD-MPC variants	Compact latent predictions	Value learning, actor optimization and/or planning

The categories overlap. **Learning a world model** and **choosing CEM** are separate design decisions. CEM and MPPI generally optimize prospective action sequences, whereas BPTT can train a parameterized feedback policy that executes quickly after training.

3.1 The planning objective

Let $A = (a_0, \dots, a_{H-1})$ denote a candidate action sequence. A planner solves

$$A^* = \arg\max_A \mathbb{E}_{\hat{P}} \left[\sum_{t=0}^{H-1} \gamma^t r(\hat{s}_t, a_t) \right].$$

CEM updates the mean and covariance of a candidate-action distribution after elite selection. MPPI weights sampled action perturbations by trajectory return. Either method can treat the predictive model as a black box. Online planning repeatedly evaluates this objective during execution; a learned feedback policy may reduce test-time computation at the cost of a policy-training stage.

4. Deriving pathwise gradients and BPTT

4.1 One-step total derivatives

Let the actor and frozen world model be

$$a_t = \pi_\theta(h(\hat{s}_t), g_t), \qquad \hat{s}_{t+1} = f_\omega(\hat{s}_t, a_t, \xi).$$

Set $D_t = \mathrm{d}\hat{s}_t/\mathrm{d}\theta$. Assuming the goal and domain context are held fixed,

$$\frac{\mathrm{d}a_t}{\mathrm{d}\theta} = \frac{\partial \pi_\theta}{\partial \theta} + \frac{\partial \pi_\theta}{\partial o_t}\frac{\partial h}{\partial s_t} D_t,$$
$$D_{t+1} = \frac{\partial f_\omega}{\partial s_t} D_t + \frac{\partial f_\omega}{\partial a_t}\frac{\mathrm{d}a_t}{\mathrm{d}\theta}.$$

This is the **total derivative recurrence** through the closed loop. If the starting state is sampled from the replay buffer, $D_0 = 0$; subsequent states still inherit the effects of earlier actor choices.

4.2 Differentiating a multi-step return

With smooth predicted rewards and ignoring termination for a moment,

$$\hat{J}_H(\theta) = \sum_{t=0}^{H-1} \gamma^t \hat{r}(\hat{s}_t, a_t) + \gamma^H V_\psi(\hat{s}_H).$$

The gradient becomes

$$\frac{\mathrm{d}\hat{J}_H}{\mathrm{d}\theta} = \sum_{t=0}^{H-1} \gamma^t \left(\frac{\partial \hat{r}_t}{\partial s_t} D_t + \frac{\partial \hat{r}_t}{\partial a_t}\frac{\mathrm{d}a_t}{\mathrm{d}\theta} \right) + \gamma^H \frac{\partial V_\psi}{\partial s_H} D_H.$$

Automatic differentiation implements this recurrence. Unlike PPO's likelihood-ratio estimator, the derivative relies on the learned dynamics Jacobian. It is a gradient of **surrogate return** $\hat{J}_H$, not necessarily an unbiased gradient of physical return J.

4.3 Freezing parameters without detaching the graph

Model parameters ω and actor parameters θ are optimized in alternating phases. During an actor phase, do not update ω, but retain $\partial f_\omega/\partial a$. Running the entire model in `no_grad()` incorrectly blocks the very derivative needed for BPTT.

4.4 Survival probabilities and termination

For a predicted termination probability $\hat{d}_{t+1} \in [0, 1]$, define $\hat{c}_0 = 1$ and

$$\hat{c}_{t+1} = \hat{c}_t(1 - \hat{d}_{t+1}).$$

A differentiable short-horizon objective consistent with the SuperMimic construction is

$$\boxed{\hat{J}_H(\theta) = \mathbb{E}\left[\sum_{t=0}^{H-1} \gamma^t \hat{c}_t \hat{r}_{t+1} + \gamma^H \hat{c}_H V_\psi(\hat{s}_H, g_H, \xi) \right].}$$

The correct definition depends on task reward, terminal semantics and which quantities are differentiable; it is not a universal copy-and-paste loss.

5. Short-horizon actor–critic and model bias

Long unrolled Jacobian products can explode or vanish. Learned state errors compound over time, while contact switches can destabilize local sensitivities. Truncating the backward horizon to H limits the path length; a learned value function approximates the remaining return:

$$G_t^{(H)} = \sum_{j=0}^{H-1} \gamma^j r_{t+j+1} + \gamma^H V_\psi(s_{t+H}).$$

True terminal states require masking the tail; time-limit truncations may still bootstrap, depending on the task contract. A critic can be fitted using TD(λ)- or GAE-style imagined targets. **Do not conflate imagined returns with evaluation returns in the actual physics simulator.**

SHAC (Short-Horizon Actor-Critic) differentiates through a **differentiable physics simulator**. SuperMimic differentiates through a **learned model trained from ordinary simulator transitions**. They share short-horizon actor–critic principles without requiring the same simulation interface.

5.1 Failure modes

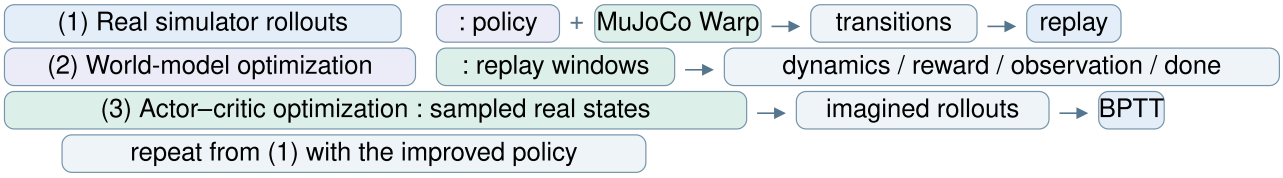
- **Model bias**: the model underestimates slip or impact penalties and the actor exploits inaccuracies;
- **Gradient bias**: low state MSE conceals incorrect $\partial f_\omega / \partial a$;
- **Distribution shift**: the actor visits regions absent from the replay buffer and receives inflated imagined return.

Practical mitigations include periodic resampling, action regularization, shorter imagination, ensemble uncertainty thresholds, early stopping and contact-stratified validation.

6. SuperMimic: a shared dynamics model for whole-body tracking

SuperMimic (2026) studies humanoid whole-body motion tracking. Rather than repeatedly training large model-free policies across motion clips, it amortizes learning through a shared model of the humanoid's dynamics and improves tracking policies via short imagined trajectories.

6.1 The alternating three-stage algorithm



Reported design choices include an **eight-step autoregressive world-model fit**, a **32-step actor imagination horizon** and terminal critic bootstrapping. Model weights remain frozen during the actor's BPTT phase; repeated collection from the rigid-body simulator keeps the surrogate aligned with the evolving policy.

6.2 Input/output contract

- Inputs: simulator state s_t, actually executed action a_t, domain randomization context ξ;
- Outputs: successor state $\hat{s}_{t+1}$, plus task-relevant reward, clean observation, and termination predictions (some heads may be analytic);
- Goal: time-aligned motion reference or future command g_t for actor and critic;
- Deployment: the trained actor must not depend on simulator-only privileged variables that real sensors cannot provide.

6.3 Reading the comparison with SONIC correctly

The authors report that roughly three hours on one H100 surpassed the return of a released SONIC checkpoint trained for around seven days on 128 L40 GPUs, with further reductions in tracking failures after longer SuperMimic training. **The hardware, training-motion data and engineering setup are not held constant**. This is not a causal, matched-resources ablation of BPTT versus PPO. Report GPU-hours, wall-clock time, actual simulator transitions and the complete evaluation protocol separately.

6.4 What is and is not demonstrated

The original paper evaluates **humanoid whole-body tracking**, including a physical Unitree G1 demonstration. It does **not** establish physical in-hand cube reorientation on a dexterous hand. That extension requires object dynamics, contact-event modeling and deployable visual/tactile state estimation.

7. Contact gradients in manipulation

Rigid-body interactions alternate between separation, impact, sticking and sliding. Nonsmooth changes at mode boundaries mean the learned smooth surrogate provides an optimization proxy rather than the exact derivative of each contact event. A precise local state prediction therefore does not automatically imply a useful policy update.

At smooth points, compare model error and local finite-difference sensitivities. Near impacts, test whether small action perturbations improve **actual simulator return** rather than requiring a globally defined classical derivative.

Suggested hand-model inputs: joint positions/velocities, object pose/twist, target orientation, recent actions, and available friction/actuator context. Privileged training targets and deployable observations must be distinguished.

8. Reproducible training and evaluation

Stage	Measurements	Failure response
A. Simulator validation	Inertia, joint PD, control rate, contact behavior	Repair robot asset or actuator settings
B. Model-free baseline	PPO success, simulator transitions, GPU-hours	Fix seeds and make baseline repeatable
C. Dynamics learning	1/8/32-step prediction, contact errors	Multi-step fitting, contact-aware sampling
D. BPTT + value	Real-simulator return gain, gradient norms, horizon sensitivity	Shorter horizon or fewer model steps
E. Generalization	Held-out commands, initial states, physics	Broaden coverage and randomization
F. Sim-to-real boundary	Estimation delay, drops, safety limits	Re-identify/calibrate before hardware trials

8.1 Required ablations

1. PPO without a world model versus MBRL at matched **real-simulator transition** budgets;
2. BPTT with versus without terminal critic;
3. $H \in \{8, 16, 32, 64\}$ rather than assuming longer is always better;
4. Single-step versus eight-step autoregressive dynamics fitting;
5. Frozen-buffer actor learning versus cyclic collection and model refresh;
6. Termination prediction, contact-biased sampling and model ensembles.

8.2 Reporting rules

Separate physical-simulator and imagined transitions; time simulator collection, model fitting and actor/critic updates; state GPU type/count and seeds with uncertainty intervals. For contact-rich tasks, mean return alone cannot replace drop rate, orientation error, and success rate.

9. Study route and references

Recommended path: original #6 Dynamics → #7 Simulation → #9 PPO and #10 SAC → Sections 1–5 here → SuperMimic in Section 6 → the new Dexterous World Models case study. For online planning use PETS/CEM/MPPI; for video world models use the original WAM tutorial.

Self-check:

1. Derive $\mathrm{d}\hat{s}_2/\mathrm{d}\theta$ and identify the effect of the first action on the second state.
2. Which computation graph supplies the policy gradient in PPO, SHAC and SuperMimic, respectively?
3. How can low one-step prediction MSE coexist with a poor actor update direction?
4. Why is a 32-step imagination horizon compatible with eight-step world-model fitting?
5. Name three measurable failure outcomes in dexterous cube reorientation.

Primary sources:

- SuperMimic: Scaling Humanoid Whole-Body Control with One Differentiable World Model (2026)
- Accelerated Policy Learning with Parallel Differentiable Simulation / SHAC (2022)
- PETS: Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models (2018)
- Dreamer: Dream to Control (2019)
- TD-MPC: Temporal Difference Learning for Model Predictive Control (2022)
- Dex-Supermimic: independent community hand adaptation (not the original authors' training release)

Attribution and limitations: This is an independent course synthesis, not an official SuperMimic tutorial. No new benchmarks were claimed to have been executed. New original text follows CC BY-NC-SA 4.0; existing course chapters, papers and external software retain their licenses.